

RELAZIONE HOMEWORK 2 DSBD

Luca Eterno – Rosario Grasso

I. Abstract

Il secondo homework estende il sistema distribuito sviluppato nel primo progetto, introducendo meccanismi di comunicazione asincrona, fault tolerance, notifiche automatiche e un gateway HTTP centralizzato.

In particolare, il progetto evolve verso una architettura event-driven, basata su Apache Kafka, in cui:

- il Data Collector agisce come producer di eventi;
- l'Alert System consuma tali eventi, verifica condizioni di soglia definite dagli utenti e produce notifiche;
- l'Alert Notifier System consuma le notifiche e le recapita via email.

Sono inoltre introdotti:

- un Circuit Breaker per proteggere le chiamate alle API esterne OpenSky;
- un API Gateway basato su Nginx, che centralizza l'accesso ai microservizi;
- Gestione delle soglie utente;

II. Nuove funzionalità

Tra le nuove funzionalità da implementare si trovano le soglie sul numero di voli che ogni utente può impostare per ogni aeroporto di suo interesse. Essendo legate all'aeroporto di interesse le soglie sono state aggiunte come nuovi campi della tabella `user_airports` che nel precedente homework accoglieva le associazioni tra gli utenti e i loro aeroporti di interesse. Poiché i voli presenti nel sistema vengono aggiornati ogni 12 ore le soglie sono state intese come in riferimento a tale lasso di tempo. Quindi ogni 12 ore, se il numero di voli atterrati nelle precedenti 12 ore ricade fuori dalle soglie impostate l'utente riceverà una mail di notifica. Per testare questa funzionalità in qualsiasi momento è stato aggiunto l'endpoint `refresh_flights_for_airport`, che simula il comportamento dell'aggiornamento periodico.

Un'altra funzionalità implementata è quella del circuit breaker. Il circuit breaker protegge il sistema dai fallimenti delle API di OpenSky restituendo al client un fallimento rapido quando il servizio esterno non risponde. Poiché un singolo fallimento potrebbe essere sintomo di un problema passeggero si è implementato il circuit breaker in modo che solo dopo una certa quantità di fallimenti consecutivi (5) esso passa allo stato di `OPEN` per certo intervallo di tempo (30s).

Apache Kafka è stato aggiunto per fare da message broker tra Data Collector, Alert System e Alert Notifier System. Nel contesto dell'aggiornamento periodico dei voli degli aeroporti di interesse, Data Collector invia un messaggio per ogni aeroporto aggiornato sul topic "`to-alert-system`", in questo modo il consumer Alert System può iniziare a elaborare il messaggio mentre Data Collector prepara i messaggi successivi. Un eventuale fallimento durante l'elaborazione del messaggio per un certo aeroporto, magari durante la chiamata all'API di OpenSky, si ripercuoterà solo su tale messaggio. Inoltre conoscendo in anticipo il numero degli aeroporti sarebbe semplice mappare un aeroporto in una partizione. In questo modo avremmo i messaggi ordinati per ogni aeroporto e migliorerebbero scalabilità e affidabilità. Infatti potremmo affidare a diverse copie di Alert System diversi aeroporti.

In entrambi i servizi consumer (Alert System e Alert Notifier System), il commit degli offset avviene manualmente solo dopo il completamento con successo dell'elaborazione del messaggio. In questo modo si ottiene at-least-once, evitando la perdita di eventi in caso di crash del servizio e rendendo il sistema più robusto rispetto a failure temporanei.

Infine l'API gateway NGINX centralizza l'accesso al sistema e rende trasparente la topologia interna. I microservizi non espongono più direttamente le proprie porte HTTP verso l'esterno. Questo approccio migliora la sicurezza complessiva del sistema e rafforza l'isolamento tra componenti.

NGINX è configurato per ascoltare sia su HTTP (porta 80) sia su HTTPS (porta 443), con supporto a certificati SSL, che essendo auto-firmati sono stati caricati sul repository al fine di facilitare il testing.

III. Schema Architetture

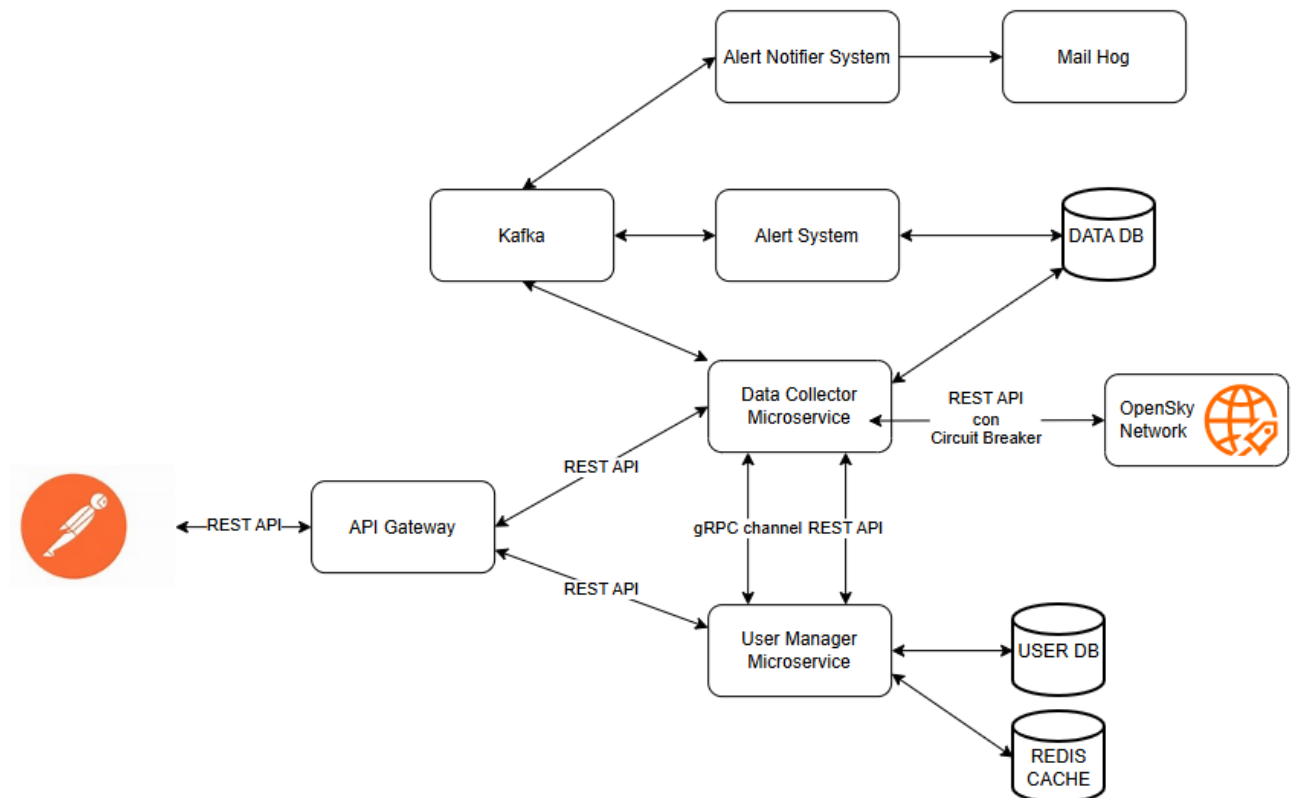


Figura 1: Schema Architetture

Il diagramma architetture mostrato in figura 1 aggiunge rispetto al homework 1:

- **API Gateway** (NGINX) che smista le richieste REST ai microservizi.
- **Data Collector microservice:**
 - Espone un API REST per permettere ad un utente di aggiornare le proprie soglie.
 - Espone un API REST di debug per ottenere informazioni sui failure e sul funzionamento del circuit breaker.
 - Comunica con Alert System tramite kafka per segnalare l'aggiunta nel `data_db` di nuovi voli per gli aeroporti di interesse. In particolare scrive un messaggio come producer kafka nel topic "to-alert-system" per ogni aeroporto di interesse indicando il numero di voli presenti nel db arrivati nel periodo di tempo a cui il refresh si riferisce.
- **Alert System:**
 - Comunica con il Data Collector tramite kafka per verificare se il numero di voli supera le soglie dell'utente. Alert System consuma le notifiche che il Data Collector gli manda tramite kafka sul topic "to-alert-system" ottenendo il numero di voli di interesse per un singolo aeroporto.
 - Comunica con il `data_db` per ottenere le soglie degli utenti riguardanti un certo aeroporto.
 - Comunica con il Alert Notifier System tramite kafka per notificare agli utenti il superamento di una soglia in riferimento ad un certo aeroporto. Alert System produce una notifica con tutte le informazioni utili e lo invia come producer kafka nel topic "to-notifier"

- **Alert Notifier System:**

- Comunica con Alert System tramite kafka per notificare ad un utente se il numero di voli supera le soglie impostate. Alert Notifier System consuma le notifiche che il Alert System gli manda tramite kafka sul topic “to-notifier” ottenendo tutte le informazioni utili a mandare una mail all’utente.
- Comunica con il server SMTP Mail Hog per testare l’invio di mail agli utenti.

- **Mail Hog:**

- Intercetta le email inviata da Alert Notifier System a fini di testing.

IV. Interazioni tra i componenti e API implementate

Questa sezione descrive nel dettaglio i flussi di comunicazione:

Client → API gateway

Il client (Postman) invia richieste HTTP e HTTPS verso i microservizi e l’API gateway le smista

API gateway → Data Collector

Il client (Postman) interagisce con Data Collector tramite REST attraverso l’ API gateway per:

- **add_user_airport** - POST /user/airports aggiornata rispetto all’homework 1 per aggiungere le soglie → body JSON (email, password, airport_code, high_value(opzionale), low_value(opzionale)):
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica le credenziali dell’utente via gRPC su User Manager;
 - Inserisce il nuovo interesse in data_db con le soglie
- **update_user_airport_thresholds** - POST /user/airports per aggiornare le soglie di un utente associate ad un aeroporto → body JSON (email, password, airport_code, high_value(opzionale), low_value(opzionale)):
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Verifica le credenziali dell’utente via gRPC su User Manager;
 - Aggiorno il data_db con le nuove soglie
- **refresh_flights_for_airport** - POST /airport/refresh-flights aggiornata rispetto all’homework 1, adesso non aggiorna i voli di un solo aeroporto a richiesta ma di tutti gli aeroporti di interesse, per simulare il comportamento del thread → body JSON (hours (24h se assente), direction (both se assente))
 - Verifica i campi della request e gestisce eventuali eccezioni;
 - Ottiene tutti gli aeroporti di interesse tramite query su data_db
 - Per ogni aeroporto:
 - Chiamata OpenSkyNet per ottenere i voli delle ultime hours ore
 - Accede a data_db per ottenere il numero di voli presenti nel database che si riferiscono alle ultime hours ore.
 - Manda una notifica kafka

Il client riceve in risposta un JSON contenente dettagli sull’operazione e status code.

Alert System → Kafka

Alert System interagisce con Kafka come consumer per il topic “to-alert-system” e come producer per il topic “to-notifier”

Alert Notifier System → Kafka

Alert Notifier System interagisce con Kafka come consumer per il topic “to-notifier”.

V. UML

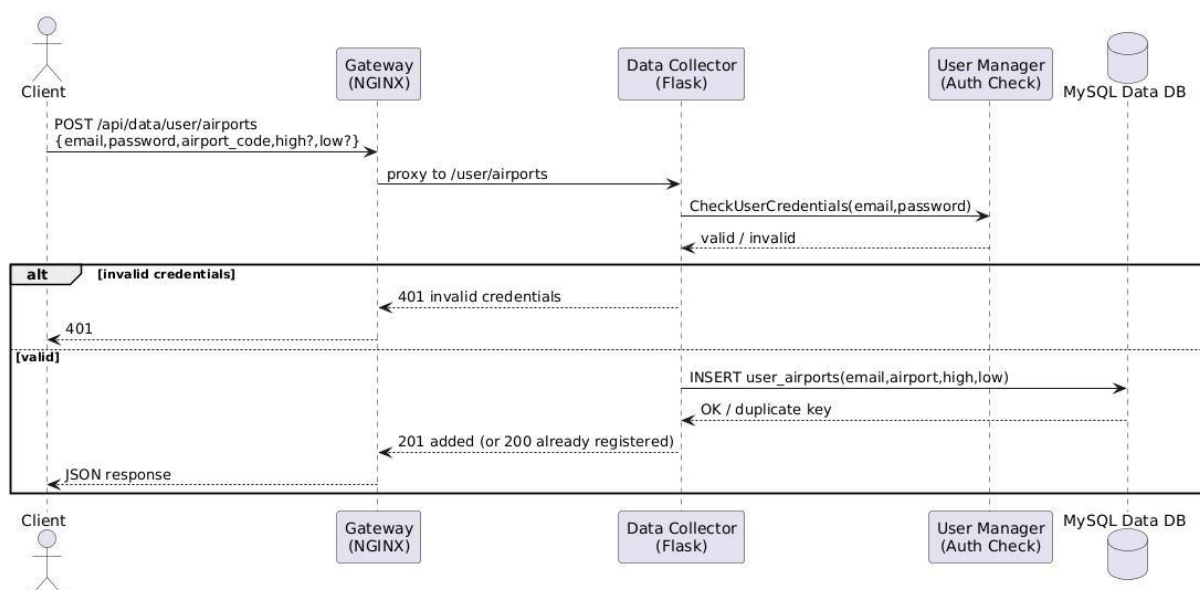


Figura 1: UML aggiunta interessi utente

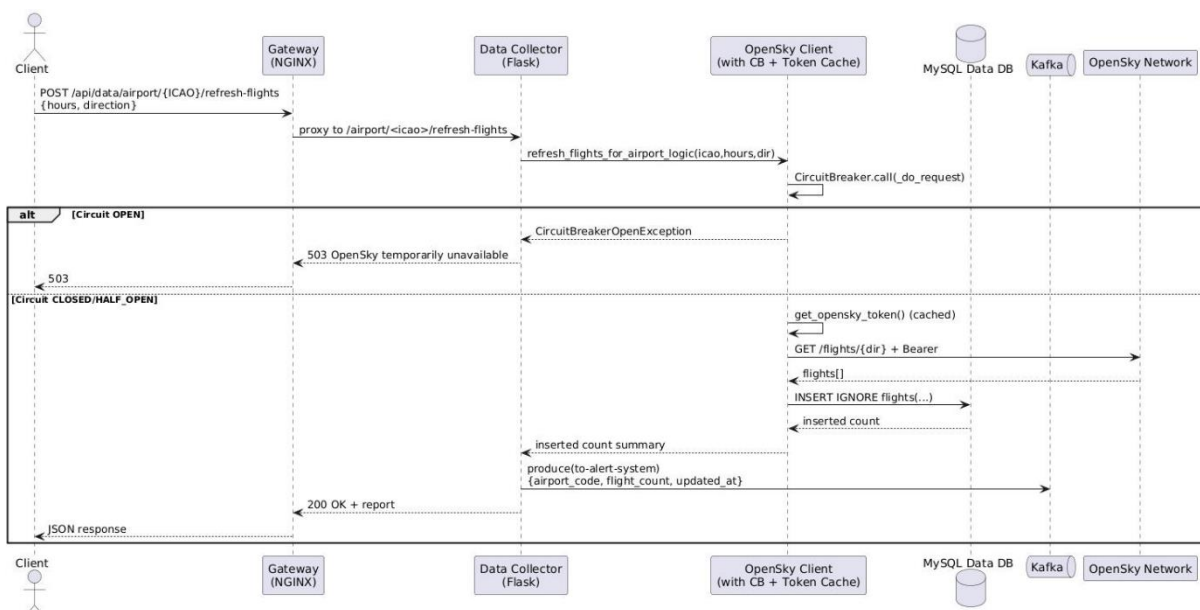


Figura 2: UML aggiornamento aeroporti di interesse

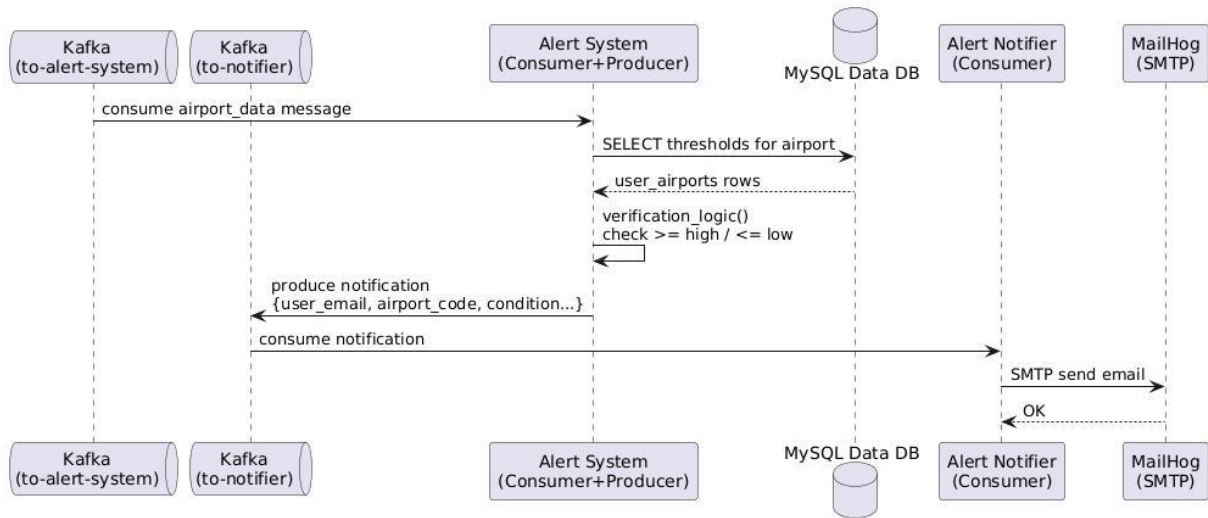


Figura 3: UML notifiche Kafka

VI. Generalità sulla soluzione proposta e conclusioni

L'obiettivo principale di questo homework è stato quello di introdurre comunicazione asincrona, gestione delle soglie utente, notifiche automatiche e meccanismi di tolleranza ai guasti. In particolare:

- **Adozione di un'architettura event-driven.**

L'introduzione di Apache Kafka ha permesso di disaccoppiare temporalmente e logicamente il Data Collector dai servizi di elaborazione delle notifiche. Ogni aggiornamento dei dati dei voli genera eventi indipendenti, che possono essere consumati, elaborati e notificati in modo asincrono, migliorando robustezza e scalabilità complessiva del sistema.

- **Gestione delle soglie utente e notifiche automatiche**

Il sistema consente a ciascun utente di definire soglie personalizzate sul numero di voli per ogni aeroporto di interesse. Queste soglie vengono verificate automaticamente ad ogni ciclo di aggiornamento, senza intervento diretto del client.

- **Separazione netta delle responsabilità tra microservizi**

Ogni componente svolge un ruolo ben definito:

- il Data Collector raccoglie i dati;
- l'Alert System applica la logica di business sulle soglie;
- l'Alert Notifier System si occupa esclusivamente della consegna delle notifiche.

- **Affidabilità e tolleranza ai guasti**

Sono stati introdotti più livelli di protezione:

- il Circuit Breaker protegge il sistema da fallimenti ripetuti delle API OpenSky, evitando overload e degradazione a cascata;
- il commit manuale degli offset Kafka garantisce semantica at-least-once, prevenendo la perdita di eventi in caso di crash temporanei dei consumer;

- **Centralizzazione dell'accesso tramite API Gateway**

L'adozione di NGINX come gateway consente di esporre un unico punto di ingresso al sistema, nascondendo la topologia interna dei microservizi.

- **Scelte progettuali non esplicitamente richieste ma essenziali**

Durante lo sviluppo sono state introdotte alcune funzionalità aggiuntive ritenute fondamentali per la coerenza del sistema:

- la gestione della cancellazione delle associazioni utente–aeroporto in caso di eliminazione dell'utente;
- l'indicizzazione delle tabelle critiche per migliorare le performance delle query temporali;
- la produzione di eventi Kafka granulari (uno per aeroporto), che favorisce parallelismo e scalabilità futura.